Cell 1 - code

```
import pandas as pd
import numpy as np
import nltk
import re
import matplotlib.pyplot as plt
import seaborn as sns
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Cell 2 - code

```
nltk.download('stopwords')
nltk.download('wordnet')
```

Cell 3 - code

```
df = pd.read_csv(r"C:\Users\AB\Downloads\archive (16)\bbc-text.csv")

df.head()
```

Cell 4 - code

```
print(df.shape)
```

Cell 5 - code

```
print(df.info())
```

Cell 6 - code

```
print(df['category'].value_counts())
```

Cell 7 - code

```
stop_words = set(stopwords.words('english'))

lemmatizer = WordNetLemmatizer()
```

Cell 8 - code

```
def clean_text(text):

text = text.lower()

text = re.sub(r'[^a-zA-Z]', ' ', text)

words = text.split()

words = [lemmatizer.lemmatize(word)
for word in words
if word not in stop_words]

return " ".join(words)
```

Cell 9 - code

```
df['clean_text'] = df['text'].apply(clean_text)
```

Cell 10 - code

```
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
```

Cell 11 - code

```
model = MultinomialNB()

model.fit(X_train, y_train)
```

Cell 12 - code

```
y_pred = model.predict(X_test)
```

Cell 13 - code

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
```

Cell 14 - code

```
sns.heatmap(cm,
annot=True,
fmt='d',
cmap='Blues',
xticklabels=model.classes_,
yticklabels=model.classes_)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()
```

Cell 15 - code

```
news = [
"India defeated Australia by five wickets in the cricket world cup final."
]

news_clean = [clean_text(i) for i in news]

news_vector = tfidf.transform(news_clean)

prediction = model.predict(news_vector)

print("Category:", prediction[0])
```